HDU R2 Solution

Mountain of Faith Round I

Suzuran

Mountain of Faith Team

March 26, 2026

A

Problem Statement

Given two non-negative integers s and t in binary form, count the number of pairs (a, b) such that $a \times b = (a \texttt{ or } b) \times (a \texttt{ and } b)$, where both a and b are integers in $[s, t]$. Here, `or` denotes bitwise OR, and `and` denotes bitwise AND.

Output the answer modulo 998244353. $1 \leq |S|, |T| \leq 5 \times 10^5$.

Observe that $a + b = (a \texttt{ or } b) + (a \texttt{ and } b)$. Let $x = (a \texttt{ or } b)$ and $y = (a \texttt{ and } b)$. Solving the system $\begin{cases} x + y = a + b \\ xy = ab \end{cases}$ gives either $\begin{cases} x = a \\ y = b \end{cases}$ or $\begin{cases} x = b \\ y = a \end{cases}$. Therefore, after assuming $a < b$, the condition in the problem is equivalent to $a = (a \texttt{ and } b)$. Let the number of such ordered pairs be S. By symmetry, the case $a > b$ also contributes S, and the case $a = b$ always satisfies the requirement, contributing $t - s + 1$ pairs. Hence the final answer is $2S + t - s + 1$.

We now compute S. Define $f(i, j, k, l)$ as follows: we process bits from high to low up to bit i; j indicates whether the current minimum possible value of a is always at least s; k indicates whether the current maximum possible value of b is always at most t; and l indicates whether the current maximum possible value of a is always less than the current minimum possible value of b. Then we can derive the transitions by case analysis. The time complexity is $O(n)$.

B

Problem Statement

Given n rays parallel to the coordinate axes. For any two overlapping rays, the length of their common part is infinite. Determine the minimum number of points that must be chosen so that every ray contains at least one chosen point.
$1 \leq n \leq 300$.

First, if ray A is completely contained in ray B, then any point on A is also on B. Therefore, ray B can be ignored during the computation.

After handling this, one can show that any point lies on at most two rays, and if it lies on two rays, then one must be horizontal and the other vertical. This is because after the above reduction, through any point there can be at most one horizontal ray and at most one vertical ray.

Construct a graph in which each ray is represented by a vertex, and connect two vertices if the corresponding rays can be covered by the same point. By the above analysis, this graph is bipartite. Clearly, the more pairs of rays we can cover with one point, the better the solution, so we should choose as many pairwise vertex-disjoint edges as possible. Therefore, the final answer is the number of vertices minus the size of a maximum matching. Kuhn's algorithm, Hopcroft–Karp, or max flow can all be used. The time complexity is $O(n^3)$ or $O(n^{2.5})$, both of which are sufficient.

C

Problem Statement

In each move, Alice may choose any one number. In each move, Bob must choose the first x numbers; if fewer than x numbers remain, he takes all of them.

Alice moves first, and they alternate until the sequence becomes empty. Alice wants to maximize the sum of the numbers she picks. Compute this value for every x.

$2 \leq n \leq 2 \times 10^5, 1 \leq a_i \leq 10^9$.

Let us re-characterize the original problem. We find that the numbers chosen by Alice must satisfy the following: among the first $x+1$ numbers, she can choose at most 1; among the first $2(x+1)$ numbers, at most 2; among the first $3(x+1)$ numbers, at most 3; and so on. In total she must choose exactly $\lfloor \frac{n}{x+1} \rfloor$ numbers. Moreover, any set of numbers satisfying the above characterization is a valid choice set for Alice. This is both necessary and sufficient.

This allows us to solve a single fixed x in $O(n \log n)$ time using a regret-greedy strategy. However, optimizing this directly for all x is difficult.

Reversing the perspective and continuing the characterization, we obtain the following equivalent description of Alice's valid choices: let $k = \lfloor \frac{n}{x+1} \rfloor - [(x+1) \mid n]$. Then among the last $n - k \cdot (x+1)$ numbers, at least 1 must be chosen; among the last $n - (k-1) \cdot (x+1)$ numbers, at least 2 must be chosen; among the last $n - (k-2) \cdot (x+1)$ numbers, at least 3 must be chosen; and so on.

That is, first take the maximum among the last $n - k \cdot (x+1)$ numbers and delete it; then take the maximum among the last $n - (k-1) \cdot (x+1)$ numbers and delete it; then take the maximum among the last $n - (k-2) \cdot (x+1)$ numbers and delete it; and so on.

Use a segment tree to speed this process up. For a single value of x, we perform $\lceil \frac{n}{x+1} \rceil$ range-maximum queries and $\lceil \frac{n}{x+1} \rceil$ point updates setting values to $-\infty$. After finishing one value of x, we perform another $\lceil \frac{n}{x+1} \rceil$ point updates to restore the original sequence.

For multiple values of x, the resulting complexity forms a harmonic series, so we can apply this algorithm directly.

Thus the complexity is $O(\sum_{x=1}^{n-1} \lceil \frac{n}{x+1} \rceil \log n) = O(n \log^2 n)$.

D

Problem Statement

Given two non-negative integers n and k, construct n positive integers such that the product of these numbers is greater than their sum, and the difference between the two is exactly k.
$2 \le n \le 2 \times 10^5, 0 \le k \le 10^7, 1 \le x_i \le 10^8$.

Take $n-2$ ones, one 2, and one $n+k$. These n numbers work.

E

Problem Summary

Given n circles whose radii are integers not exceeding 30, and whose centers are all lattice points. For each of m queries, you are given a rectangle whose four vertices are all lattice points. Determine how many circles have non-empty intersection with the rectangle.

$2 \leq n \leq 1.5 \times 10^6, 1 \leq m \leq 10^4, 1 \leq x_i, y_i, p_i, q_i \leq 10^9$.

Consider the condition under which a circle of radius R intersects a rectangle. We divide it into the following three cases:

1. The center of the circle is inside the rectangle or on its boundary. Then the statement is obviously true.
2. The center is outside the rectangle, but the perpendicular dropped to the rectangle along the x- or y-direction intersects the rectangle. Suppose the distance from the center to the foot of that perpendicular is d. Then the statement holds iff $d \leq R$.
3. The center is outside the rectangle, and neither the vertical nor horizontal perpendicular to the rectangle intersects it. Suppose the distance from the center to the nearest vertex of the rectangle is d. Then the statement holds iff $d \leq R$.

According to the above, for a circle of radius R centered at (p_i, q_i), its intersection with the rectangle whose lower-left and upper-right corners are (x_1, y_1) and (x_2, y_2) is non-empty iff one of the following conditions holds:

1. $y_1 \le q_i \le y_2$ and $x_1 - R \le p_i \le x_2 + R$;
2. $y_2 < q_i \le y_2 + R$ and $x_1 - \sqrt{R^2 - (q_i - y_2)^2} \le p_i \le x_2 + \sqrt{R^2 - (q_i - y_2)^2}$;
3. $y_1 - R \le q_i < y_1$ and $x_1 - \sqrt{R^2 - (y_1 - q_i)^2} \le p_i \le x_2 + \sqrt{R^2 - (y_1 - q_i)^2}$.

This is equivalent to performing 2D orthogonal range counting inside a rounded rectangle.

Notice that the coordinates in this problem are discrete. Therefore, for Cases 2 and 3, we can brute-force all possible values of q, find the corresponding x-coordinate intervals, and thus split the upper and lower rounded parts into R line segments each (or equivalently, rectangles of width 0). When counting the answer, we simply sum the answers for all $2R+1$ rectangles corresponding to the three cases.

Hence, we may first enumerate the radius r. For each query rectangle, perform the above decomposition, and then do 2D range counting between those rectangles and the points with radius r. The time complexity is $O((n+mR^2)\log(n+mR))$, which is sufficient for this problem. We set the time limit to twice that of this solution without especially aggressive constant optimization.

Bonus: It can be reduced to $O(n\log n + mR\log n + mR^2\log_B V)$. If radix sort is regarded as linear, then the complexity becomes $O(n\log n + mR\log n + mR^2)$.

F

Problem Statement

Given an array $a_1, a_2 \cdots, a_n$ of length n, after rearranging it, maximize the number of indices i $(1 \le i \le n)$ such that $a_i = i$. $1 \le n \le 5 \times 10^5$.

Observe:

- Any number greater than n can never contribute.
- For each number not exceeding n, no matter how many times it appears, it can contribute exactly once, and only at its unique position distinct from all other numbers.

Therefore, we only need to count how many distinct numbers in $[1, n]$ appear in the array. The time complexity is $O(n)$.

G

Problem Summary

Define the weight of a multiset as the XOR of all its elements. Alice and Bob alternately delete one number; the player whose move makes the weight become 0 loses.
For all subarrays of the non-negative integer sequence $\{x_i\}_{i=1}^{n}$, count how many of them are winning for Alice and how many are winning for Bob.
$1 \leq n, x_i \leq 2 \times 10^5$.

Conclusion first.

For the game on a multiset S, the first player wins if and only if $|S|$ is even and the weight of S is not 0.

Initially, if the weight of S is 0, then the second player certainly wins. Consider the move immediately before a player loses, and the weight of S at that moment. Suppose the previous move removed an element x. Then the weight of S before that move was also x. If the elements of S before that move were <u>not all identical</u>, then there exists some element $y \neq x$. Removing y instead makes the weight become $x \oplus y \neq 0$. Deleting y keeps the game going, so this is strictly better than deleting x.

Therefore, in the position immediately before the final move, all elements of S must be identical. They are still identical after that move. To make the previous weight non-zero and the next weight zero while all elements are identical, the terminal multiset must consist of an even number (possibly zero) of equal elements, and that element must be non-zero.

Starting from a multiset S with non-zero weight, the final multiset always has even size, so the parity of the number of moves is fixed and depends only on the parity of the initial size of S: if $|S|$ is even, the first player wins; if $|S|$ is odd, the second player wins.

We now count all intervals winning for the first player, i.e., intervals whose length is even and whose XOR is non-zero.
First assume that every even-length interval has non-zero XOR and count them. This is easy. Then subtract those intervals whose XOR is zero and whose length is even.
Given the original sequence $x_1, x_2, \cdots, x_n$, define the prefix XORs $s_0 = 0$ and $s_i = s_{i-1} \oplus x_i$. Then the XOR of interval $[l, r]$ is $s_{l-1} \oplus s_r$.

We need to count intervals $[l, r]$ $(1 \le l \le r \le n)$ such that $r - l + 1$ is even and $s_{l-1} \oplus s_r = 0$. This is equivalent to counting pairs $[l, r]$ $(0 \le l < r \le n)$ such that $r - l$ is even and $s_l = s_r$. Notice that when $r - l$ is even, l and r have the same parity. Group together all equal values of s_i with even i $(0 \le i \le n)$, and separately group together all equal values of s_i with odd i $(0 \le i \le n)$. For any two values s_i and s_j in the same group with $i < j$, the interval $[i + 1, j]$ contributes 1. Values from different groups contribute nothing. If such a group has size m, then it contributes $\frac{1}{2} \cdot m(m - 1)$.
The complexity is $O(n)$ or $O(n \log n)$, depending on the implementation.

H

Problem Statement

There are n integers $a_1, a_2, \ldots, a_n$ arranged clockwise on a circle. It is known that their sum is 0.

You need to choose a starting position k such that, taking n consecutive numbers clockwise starting from a_k, all of their prefix sums are non-negative.

It can be proved that such a k always exists. If there are multiple valid choices, output the smallest one.

$1 \leq n \leq 2 \times 10^5, 0 \leq |a_i| \leq 10^9$.

Let the prefix sums be $S_0 = 0$ and $S_i = \sum_{j=1}^{i} a_j$ $(1 \le i \le n)$. Since the total sum is 0, we have $S_n = 0$.

Conclusion

Let m be the smallest index at which the minimum value of the sequence $S_0, S_1, \ldots, S_{n-1}$ is attained. Then the answer is $k = m + 1$.

For any l with $0 \le l \le n - 1$, consider the prefix sum of length $l + 1$ starting from $k = m + 1$, namely $\sum_{i=0}^{l} a_{k+i}$.

Turn the cyclic indices into ordinary prefix-sum differences:

- If $m + l + 1 \leq n$, then $\sum_{i=0}^{l} a_{k+i} = S_{m+l+1} - S_m \geq 0$, because S_m is the minimum prefix sum.
- If $m + l + 1 > n$, let $u = m + l + 1 - n$ $(1 \leq u \leq n - 1)$. Then the sum along this wrapped segment on the circle equals $(S_n - S_m) + S_u = (0 - S_m) + S_u = S_u - S_m \geq 0$, again because S_m is the minimum value (and $S_n = 0$).

Therefore all such prefix sums are non-negative, so $k = m + 1$ is a valid starting position.

Next we show that $k = m + 1$ is the smallest valid starting position.

- If $m = 0$, then $S_i \geq 0$ for all $0 \leq i \leq n$, so $k = 1$, which is obviously the smallest possible starting point.
- If $m > 0$, then since $S_0 = 0$ and S_m is the first position attaining the minimum prefix sum, we must have $S_m < 0$. For any $k' \leq m$, consider the prefix sum from k' to position m: $\sum_{i=0}^{m-k'} a_{k'+i} = S_m - S_{k'-1}$. Because m is the first occurrence of the minimum prefix sum, we have $S_{k'-1} > S_m$. Hence $S_m - S_{k'-1} < 0$. Therefore a negative prefix sum must appear when starting from k', so k' is not valid. Thus every $k' \leq m$ is invalid, and any valid starting point must satisfy $k \geq m + 1$.

Hence the answer is $m + 1$, and the time complexity is $O(n)$.

I

Problem Statement

Given an $n \times m$ grid, you want to color some cells black so that the following two conditions are satisfied:

- No two adjacent cells are both black;
- The final pattern must have a horizontal or vertical axis of symmetry.

Determine the maximum number of cells that can be colored black. $1 \leq n, m \leq 10^9$.

A classical result on grid graphs states that at most $\lceil nm/2 \rceil$ pairwise non-adjacent cells can be chosen.
If at least one of n, m is odd, alternating coloring achieves this upper bound, and the pattern also has an axis of symmetry perpendicular to the odd side.
If both n and m are even, then the cells on both sides of the symmetry axis must remain uncolored. After enumerating the orientation of the symmetry axis, we can alternately color the remaining part. The answer is $nm/2 - \min\{n, m\}$.

J

Problem Statement

Given x, k, find the k-th non-empty sequence of positive integers in lexicographic order among all sequences whose sum of elements does not exceed x.
$1 \leq x \leq 10^6, 1 \leq k \leq 10^{18}$.

In the following discussion, the empty sequence is excluded.

Define $f(x)$ as the number of sequences whose element sum is exactly x. Then $f(1) = 1$. Consider $x > 1$. For every $1 \leq x' < x$, we can append the number $x - x'$ to any sequence whose sum is x' to obtain a sequence whose sum is x. Clearly, this construction is bijective, so $f(x) = \sum_{i=1}^{x-1} f(i) + 1$. We can prove that $f(x) = 2^{x-1}$.

Proof

When $x = 1$, $f(1) = 1 = 2^{1-1}$, so the statement holds. When $x = 2$, $f(2) = f(1) + 1 = 2^{2-1}$, so the statement holds.
Assume that for $k = x$ $(x \geq 2)$, we have $f(k) = 2^{k-1}$. Then

$$f(k+1) = \sum_{i=1}^{k} f(i) + 1 = f(k) + \sum_{i=1}^{k-1} f(i) + 1 = 2f(k) = 2^{(k+1)-1}.$$

Thus the statement also holds for $k + 1$, and the induction is complete.

Define $g(x)$ as the number of sequences whose element sum is at most x, then $g(x) = \sum_{i=1}^{x} f(i)$. By the formula for a geometric series, $g(x) = 2^x - 1$.

Consider the number of sequences whose first element is 1 and whose sum does not exceed x. Except for the sequence $[1]$, every such sequence can be obtained by prepending a 1 to a sequence whose sum does not exceed $x - 1$. Therefore, under the constraint that the sum does not exceed x, the number of sequences beginning with 1 other than $[1]$ is $g(x-1) = 2^{x-1} - 1$, and the number of sequences not beginning with 1 is $g(x) - g(x-1) - 1 = 2^{x-1} - 1$.

Hence, we consider whether the first element of the sequence is 1. If $k = 1$, then the sequence is $[1]$, so we can return it directly. Otherwise, if $k \leq 2^{x-1}$, then the first element is 1. After removing this leading 1, the remaining elements have sum at most $x - 1$, and we need to find the sequence ranked $k - 1$ in lexicographic order among them.

The structure of this problem is the same as the original one, so it can be solved recursively. Otherwise, if $k > 2^{x-1}$, then we need to consider the sequence ranked $k - 2^{x-1}$ among those whose first element is not 1. Notice that if we subtract 1 from the first element of each remaining sequence, their relative order does not change.

Therefore, mark that this first element has been decreased by 1. To determine the rest of the sequence, we only need to find the sequence ranked $k - 2^{x-1}$ among all sequences whose element sum is at most $x - 1$, and then add 1 back to its first element afterward. This subproblem has exactly the same structure as the original one, so it can also be solved recursively.

Clearly, during the recursion we always have $1 \leq k \leq 2^x - 1$, so when we recurse down to $x = 1$, we always have $k = 1$. Hence the recursion always terminates, and the terminal value of x is at least 1. Since each recursive step decreases x by at least 1, the time complexity is $O(x)$.

Thanks!